Express.js: Routing, Middleware, and APIs

A practical introduction to building HTTP services

Purpose: This document is a fictional, self-contained learning resource created for experimenting with PDF text extraction, chunking, embeddings, retrieval, and question answering in a Retrieval-Augmented Generation (RAG) pipeline.

Contents

1. What is Express.js?

2. Routing and HTTP methods

3. Middleware

4. Validation and error handling

5. Authentication and authorization

6. Security and operational controls

7. Testing and API design

1. What is Express.js?

Express.js is a minimal web framework for Node.js. It provides routing, middleware composition, and request/response helpers for building HTTP APIs and web applications. Express does not dictate a complete application architecture, so teams choose their own validation, database, authentication, and testing libraries.

An Express application typically defines middleware, routes, error handling, and a listener. Keep business rules in service modules instead of putting every concern into route handlers.

A minimal Express server

```
const express = require("express");
const app = express();

app.get("/health", (req, res) =&gt; {
  res.json({ status: "ok" });
});

app.listen(3000, () =&gt; {
  console.log("API listening on port 3000");
});
```

2. Routing and HTTP methods

Routes map an HTTP method and path to a handler. GET is commonly used to retrieve a resource, POST to create or trigger an operation, PUT to replace a resource, PATCH to partially update it, and DELETE to remove it. Correct semantics help clients and intermediaries understand API behavior.

Route parameters identify a resource in the path, query parameters express filters or pagination, and the request body carries submitted data. Validate all values before using them in business logic or database queries.

Route parameters and query parameters

```
app.get("/api/users/:userId", (req, res) =&gt; {
  const { userId } = req.params;
  const { includeProfile } = req.query;

  res.json({
    userId,
    includeProfile: includeProfile === "true"
  });
});
```

3. Middleware

Middleware functions can read or modify the request and response, end the request, or pass control to the next middleware. Middleware runs in registration order, so parsing and security middleware generally need to be registered before the routes that depend on them.

A middleware function should either complete the response or call next. If it does neither, the request may remain open. Error-handling middleware uses four parameters: error, request, response, and next.

Request logging middleware

```
function requestLogger(req, res, next) {
  const startedAt = Date.now();

  res.on("finish", () =&gt; {
    console.log({
      method: req.method,
      path: req.path,
      status: res.statusCode,
      durationMs: Date.now() - startedAt
    });
```

```
  });

  next();
}

app.use(requestLogger);
app.use(express.json());
```

4. Validation and error handling

Never assume request data is valid because it came from a frontend. Validate types, required fields, length limits, allowed values, and business rules on the server. Validation libraries can centralize these checks and return consistent field-level errors.

Use a predictable error response format and centralized error middleware. In Express 5, rejected promises returned by route handlers and middleware are forwarded to error handling automatically; in Express 4, asynchronous failures commonly need explicit forwarding or a wrapper. Do not expose stack traces or internal database details in production responses.

Centralized error handler

```
app.use((err, req, res, next) =&gt; {
  if (res.headersSent) {
    return next(err);
  }

  console.error(err);
  res.status(err.statusCode || 500).json({
    error: {
      message: err.expose
        ? err.message
        : "Internal server error"
    }
  });
});
```

5. Authentication and authorization

Authentication establishes who the caller is. Authorization checks what that caller is allowed to do. A valid login or SSO identity alone should not automatically grant access to every application route or department's data.

Authorization checks should be enforced on the server for every protected operation. Use role- or attribute-based rules as appropriate, verify ownership and department boundaries, and fail closed when access cannot be established. Avoid trusting role names or user IDs supplied only by the client.

Illustrative authorization middleware

```
function requireRole(requiredRole) {
  return (req, res, next) =&gt; {
    // req.user must be populated by trusted authentication middleware.
    if (!req.user || !req.user.roles?.includes(requiredRole)) {
      return res.status(403).json({ error: "Forbidden" });
    }
    next();
  };
}

app.get("/api/admin/reports",
  requireRole("admin"),
  (req, res) =&gt; res.json({ reports: [] })
);
```

6. Security and operational controls

Use TLS at the appropriate network boundary, secure cookie settings when using cookie-based sessions, rate limits for sensitive endpoints, and restrictive CORS settings when cross-origin browser access is needed. CORS is not an authentication mechanism.

Set request body limits, use Helmet or equivalent security headers, validate uploaded files, and avoid exposing stack traces. Store secrets outside source control. Log request IDs, status codes, and timings while minimizing sensitive data.

7. Testing and API design

Test successful responses, validation failures, missing authentication, insufficient permissions, and unexpected errors. Integration tests can exercise an Express app through HTTP requests without requiring a manually started production server.

Use consistent status codes and response shapes. Document required parameters, response schemas, pagination behavior, and error cases. Version an API when a breaking change cannot be made compatibly.

Glossary

Route — A mapping from an HTTP method and path to request-handling logic.

Middleware — A function that participates in the request-response pipeline.

Request parameters — Values extracted from a route path, query string, headers, or body.

Error middleware — Express middleware with the signature (err, req, res, next).

Authentication — The process of establishing a caller's identity.

Authorization — The process of deciding whether a caller may perform an action.

CORS — A browser mechanism that controls cross-origin access to responses.

Practice questions for retrieval

Use these questions to test whether your RAG pipeline retrieves the relevant passages rather than answering from general model knowledge.

1. What does Express add on top of Node.js?

2. How do route parameters differ from query parameters?

3. Why does middleware registration order matter?

4. What must middleware do to avoid leaving a request hanging?

5. How does Express 5 handle rejected promises compared with Express 4?

6. Why is SSO authentication not sufficient to authorize every route?

7. Name four controls that improve the security of an Express API.

End of sample document.